

UNIVERSIDAD
PANAMERICANA

Sistema seguro de envío de documentos empleando criptografía

Criptografía y Redes de Seguridad

Montserrat Castillo Vega	0255627
Miriam Ximena García Mendoza	0254600
José Gabriel Hernández Castresana	0253264
José Eduardo Romero Garduño	0260703
Victor Gabriel Torres Guerrero	0265786

Profesor(a): Marián López Gonzáles

Ciudad de México a 15 de julio de 2026



Índice

Índice.....	2
Resumen.....	4
1. Introducción.....	4
2. Planteamiento del problema.....	4
3. Objetivos del proyecto.....	5
3.1. Objetivo general.....	5
3.2. Objetivos específicos.....	5
4. Alcance y limitaciones del sistema.....	5
5. Arquitectura/construcción del sistema.....	6
5.1. Interfaz de usuario.....	6
5.2. Módulo de autenticación.....	6
5.3 Módulo criptográfico.....	6
5.4 Módulo de base de datos.....	7
6. Diseño de la base de datos.....	7
6.1. Tabla usuarios.....	7
6.2. Tabla llaves_publicas.....	7
6.3. Tabla paquetes_archivos.....	7
6.4. Tabla logs_auditoria.....	7
7. Funcionalidades implementadas.....	7
7.1. Registro de usuarios.....	7
7.2. Inicio de sesión.....	8
7.3. Cambio de contraseña.....	8
7.4. Generación de llaves.....	8
7.5. Envío de archivos.....	8
7.6. Recepción y descifrado.....	8
7.7. Archivos enviados.....	8
7.8. Verificación manual de firma digital.....	8
7.9. Panel de administración.....	9
7.10. Simulación de ataques desde la interfaz.....	9
8. Mecanismos criptográficos utilizados.....	9
8.1. Argon2.....	9
8.2. AES-GCM.....	9
8.3. X25519.....	9
8.4. Ed25519.....	9



9. Flujo criptográfico del envío de archivos.....	10
10. Componente matemático.....	10
10.1. X25519 y el secreto compartido.....	10
10.2. AES-GCM e integridad autenticada.....	11
11. Modelo de amenazas.....	11
12. Pruebas funcionales.....	11
12.1 Registro de usuario.....	11
12.2 Generación de llaves.....	12
12.3 Envío de archivo.....	14
12.4 Recepción y descifrado.....	15
12.5 Archivos enviados.....	15
13. Pruebas de Ataques.....	16
13.1. Modificación de Archivos (Integridad y Autenticidad).....	16
13.2. Firma Falsa (Autenticidad de Origen y No Repudio).....	16
13.3. Ataque de Repetición (Replay Attack).....	17
13.4. Contraseña Incorrecta (Mitigación de Fuerza Bruta).....	17
13.5. Robo de la Base de datos.....	18
13.6. Ataque Man In the Middle.....	19
14. Wireshark.....	20
15. Parte post cuántica.....	22
17. Conclusiones.....	23
18. Referencias bibliográficas.....	24



Resumen

El presente proyecto desarrolla un sistema seguro de envío de documentos, en el que hay un intercambio de archivos sensibles entre usuarios dentro de una organización. El sistema considera un escenario donde la red no es confiable y un atacante puede capturar tráfico, modificar mensajes, reenviar comunicaciones anteriores, suplantar usuarios o intentar obtener información almacenada en el servidor.

Se implementó una aplicación web en Streamlit con mecanismos criptográficos modernos. El sistema permite registrar usuarios, iniciar sesión, generar llaves criptográficas, enviar archivos cifrados, verificar firmas digitales, consultar archivos recibidos y enviados, administrar usuarios y registrar eventos de seguridad. La solución utiliza Argon2 para proteger contraseñas, AES-GCM para cifrado autenticado, X25519 para establecer una llave compartida y Ed25519 para firma digital.

El proyecto busca los elementos de la CIA: confidencialidad, integridad, autenticidad. Asimismo, propone protección contra repetición y trazabilidad mediante logs de auditoría. Además, se implementaron pruebas funcionales y simulaciones de ataque para comprobar que el sistema rechaza modificaciones, firmas inválidas, contraseñas incorrectas y paquetes repetidos. Finalmente, se incluye un análisis de red con Wireshark para verificar el cifrado de transporte.

1. Introducción

En un entorno organizacional, el intercambio de documentos sensibles requiere mecanismos de seguridad que protejan la información incluso cuando la red no es completamente confiable. Si los archivos viajan por un canal expuesto, un atacante podría capturar el tráfico, modificar mensajes, reenviar paquetes anteriores o intentar suplantar la identidad de un usuario.

Por esta razón, el proyecto consta de un sistema seguro de envío de documentos. Este sistema es capaz de proteger los archivos mediante cifrado, firma digital y validación de integridad. La aplicación fue desarrollada con Streamlit como interfaz principal y Python como lenguaje de implementación. El diseño integra módulos separados para autenticación, criptografía, base de datos y gestión de la interfaz.

El objetivo principal es construir una solución funcional que permita a los usuarios compartir archivos de forma segura, demostrando el uso correcto de algoritmos criptográficos y evitando prácticas inseguras como almacenar contraseñas en texto claro, reutilizar nonces o guardar llaves privadas en el servidor.

2. Planteamiento del problema

La organización del escenario necesita compartir documentos sensibles entre empleados conectados mediante una red no confiable. En este contexto, se asume



que un atacante puede observar el tráfico, modificar mensajes, reenviar comunicaciones pasadas, robar archivos almacenados o intentar adivinar contraseñas.

El problema central consiste en diseñar un sistema que mantenga protegida la información aun cuando el canal de comunicación o el almacenamiento del servidor puedan ser atacados. Para ello, el sistema debe garantizar:

- Confidencialidad del archivo.
- Integridad del contenido.
- Autenticidad del emisor.
- Protección contra ataques de repetición.
- Seguridad de credenciales y llaves.
- Registro de eventos relevantes para auditoría.

3. Objetivos del proyecto

3.1. Objetivo general

Desarrollar una aplicación que permita el intercambio seguro de archivos entre usuarios mediante cifrado autenticado, firma digital, protección de llaves, autenticación de usuarios y registro de eventos de seguridad.

3.2. Objetivos específicos

1. Implementar registro e inicio de sesión de usuarios con almacenamiento seguro de contraseñas.
2. Generar pares de llaves criptográficas para firma digital y cifrado/acuerdo de llaves.
3. Cifrar archivos utilizando AES-GCM.
4. Proteger el acceso al archivo mediante X25519.
5. Firmar digitalmente los archivos enviados con Ed25519.
6. Verificar la autenticidad e integridad de los archivos recibidos.
7. Detectar paquetes repetidos mediante identificadores únicos.
8. Registrar eventos de seguridad en logs de auditoría.
9. Probar el sistema frente a ataques de modificación, firma falsa, repetición y contraseña incorrecta.
10. Analizar el tráfico de red mediante Wireshark.

4. Alcance y limitaciones del sistema

El sistema implementado permite que un usuario se registre, inicie sesión, genere llaves, envíe archivos cifrados a otro usuario, reciba archivos, descifre contenido, verifique firmas digitales, consulte archivos enviados y gestione sus llaves privadas de forma local. Además, un usuario con rol de superusuario puede consultar logs globales y bloquear o desbloquear cuentas.



El sistema no busca reemplazar/crear una infraestructura empresarial completa de certificados digitales, sino demostrar de forma práctica los mecanismos criptográficos centrales requeridos para proteger archivos sensibles.

Quedan fuera del alcance del proyecto:

- Protección de dispositivos finales comprometidos.
- Malware instalado en la computadora del usuario.
- Robo físico de las llaves privadas descargadas.
- Gestión formal de certificados emitidos por una autoridad certificadora.
- Implementación productiva de TLS en un servidor público.
- Recuperación de llaves privadas perdidas.

5. Arquitectura/construcción del sistema

La aplicación se divide en cuatro módulos principales:

5.1. Interfaz de usuario

El archivo principal de la aplicación es `app.py`, desarrollado con Streamlit. Este archivo contiene las vistas de login, registro, panel de control, gestión de archivos, gestión de llaves y administración.

La interfaz permite al usuario realizar operaciones criptográficas sin interactuar directamente con la consola. Desde la aplicación puede subir archivos, seleccionar destinatarios, cargar llaves privadas, descifrar archivos, descargar el archivo original, descargar la firma digital, consultar archivos enviados y revisar actividad reciente.

El panel de control diferencia entre usuarios normales y superusuarios o administradores. Los usuarios normales visualizan únicamente los eventos asociados a su propia cuenta, mientras que el superusuario puede consultar la bitácora global del sistema. Esta separación reduce la exposición innecesaria de información de auditoría entre usuarios.

5.2. Módulo de autenticación

El archivo `auth.py` contiene las funciones relacionadas con usuarios y contraseñas. Este módulo permite registrar usuarios, iniciar sesión, bloquear cuentas, desbloquearlas, cambiar roles y actualizar contraseñas.

Las contraseñas no se almacenan en texto claro. En su lugar, se utiliza Argon2 para generar un hash seguro. Además, el sistema registra intentos fallidos y bloquea cuentas después de varios intentos incorrectos.

5.3 Módulo criptográfico

El archivo `crypto.py` implementa los mecanismos criptográficos principales. El sistema genera llaves Ed25519 para firma digital y X25519 para cifrado/acuerdo de



llaves. Para proteger los archivos se utiliza AES-GCM, que proporciona confidencialidad e integridad mediante una etiqueta de autenticación.

5.4 Módulo de base de datos

El archivo database.py gestiona el almacenamiento de los datos en SQLite. La base de datos contiene tablas para usuarios, llaves públicas, paquetes cifrados y logs de auditoría. Las llaves privadas no se guardan en el servidor; únicamente se almacenan llaves públicas y metadatos necesarios para el intercambio seguro.

6. Diseño de la base de datos

La base de datos está compuesta por cuatro tablas principales:

6.1. Tabla usuarios

Almacena el nombre de usuario, hash de contraseña, rol, estado de la cuenta, número de intentos fallidos y último acceso.

6.2. Tabla llaves_publicas

Guarda las llaves públicas asociadas a cada usuario. Se almacena una llave pública X25519 para cifrado/acuerdo de llaves y una llave pública Ed25519 para verificación de firmas.

6.3. Tabla paquetes_archivos

Guarda los paquetes de archivos cifrados. Esta tabla incluye el emisor, destinatario, nombre del archivo, texto cifrado, IV o nonce, llave pública efímera, firma digital, identificador único del protocolo y timestamp.

El campo nonce_protocolo es único, por lo que permite rechazar paquetes repetidos y demostrar protección contra ataques de repetición (Ataque 3).

6.4. Tabla logs_auditoria

Registra eventos relevantes del sistema, como inicios de sesión, envíos, recepciones, firmas válidas, firmas inválidas, errores de descifrado, intentos de repetición, bloqueos de cuenta y cambios de contraseña.

7. Funcionalidades implementadas

7.1. Registro de usuarios

El sistema permite crear usuarios con diferentes roles. Durante el registro, la contraseña se procesa mediante Argon2 y se almacena únicamente el hash resultante.



7.2. Inicio de sesión

El usuario puede iniciar sesión con su nombre de usuario y contraseña. Si introduce una contraseña incorrecta, el sistema incrementa el contador de intentos fallidos. Después de alcanzar el límite definido, la cuenta queda bloqueada.

7.3. Cambio de contraseña

El sistema permite cambiar la contraseña después de verificar la contraseña actual. La nueva contraseña se vuelve a procesar mediante Argon2 antes de almacenarse.

7.4. Generación de llaves

Cada usuario puede generar dos pares de llaves:

- Ed25519 para firma digital.
- X25519 para cifrado/acuerdo de llaves.

Las llaves públicas se registran en la base de datos. Las llaves privadas son descargadas por el usuario y protegidas mediante una frase de seguridad.

7.5. Envío de archivos

Para enviar un archivo, el usuario selecciona un destinatario, carga el archivo y sube su llave privada de firma. El sistema firma el archivo, genera un secreto compartido mediante X25519, cifra el contenido con AES-GCM y almacena el paquete cifrado en la base de datos.

7.6. Recepción y descifrado

El destinatario puede consultar su bandeja de entrada cifrada. Para descifrar un archivo, debe subir su llave privada de cifrado y proporcionar la frase de seguridad correspondiente. El sistema reconstruye la llave AES, valida la etiqueta de autenticación y verifica la firma digital del emisor.

7.7. Archivos enviados

El sistema incluye una vista de archivos enviados. Esta sección permite consultar el historial de documentos enviados, destinatario, fecha, identificador único, nonce de AES-GCM, tamaño cifrado y resumen de la firma digital. Por seguridad, esta vista no muestra el contenido original del archivo, llaves privadas, contraseñas ni llaves de sesión.

7.8. Verificación manual de firma digital

La aplicación permite subir un archivo y una firma digital para verificar si corresponden a un emisor esperado. El sistema obtiene la llave pública Ed25519 del emisor y valida matemáticamente la firma.



7.9. Panel de administración

El superusuario puede consultar logs globales, revisar usuarios registrados y bloquear o desbloquear cuentas. Estas acciones quedan registradas en la bitácora de auditoría con nivel crítico, ya que modifican el estado de acceso de otros usuarios.

El sistema también separa la visibilidad de los logs según el rol. Los usuarios normales solo pueden consultar eventos asociados a su propia cuenta, mientras que el superusuario puede revisar los registros globales. Esto evita que un usuario común observe información de auditoría relacionada con otros usuarios.

7.10. Simulación de ataques desde la interfaz

Además del flujo normal de envío y recepción, la aplicación incluye controles para simular ataques dentro del entorno de pruebas. Por ejemplo, después de enviar un archivo, el sistema conserva temporalmente el último paquete generado en la sesión para permitir dos pruebas principales: reinyectar el mismo paquete y simular la modificación del contenido por un intermediario.

8. Mecanismos criptográficos utilizados

8.1. Argon2

Argon2 se utiliza para proteger contraseñas. En lugar de guardar contraseñas en texto claro, el sistema almacena hashes generados con una función diseñada para resistir ataques de fuerza bruta y ataques con hardware especializado (JumpCloud, 2025).

8.2. AES-GCM

AES-GCM se utiliza para cifrar archivos. Este modo proporciona confidencialidad e integridad, ya que además de generar texto cifrado produce una etiqueta de autenticación. Si el archivo cifrado, el nonce o los datos asociados son modificados, el descifrado falla (Medium, 2024).

8.3. X25519

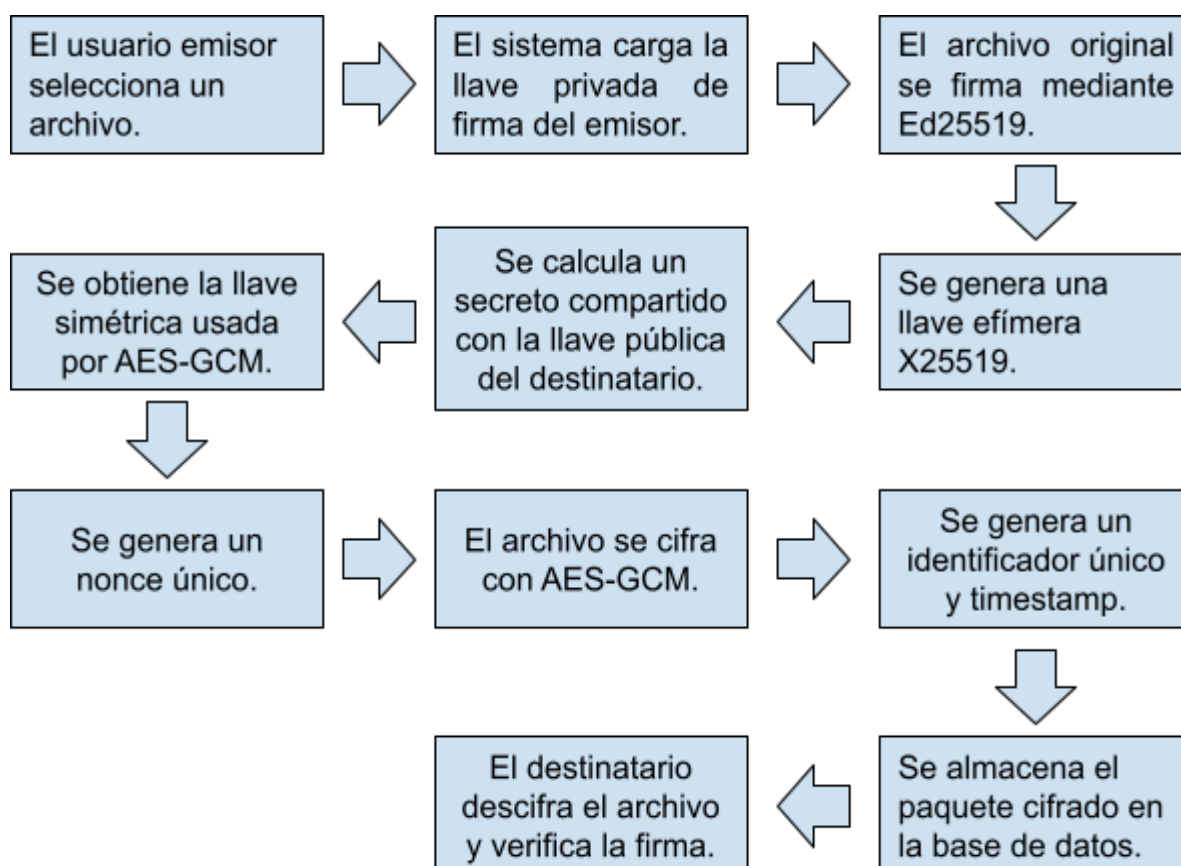
X25519 se utiliza para generar un secreto compartido entre el emisor y el receptor. El emisor genera una llave efímera y la combina con la llave pública del destinatario. El receptor puede reconstruir el mismo secreto usando su llave privada y la llave pública efímera del emisor (Hansen & Harrison, 2024).

8.4. Ed25519

Ed25519 se utiliza para firmar digitalmente los archivos. El emisor firma el contenido original con su llave privada. Posteriormente, el receptor utiliza la llave pública del emisor para comprobar que el archivo no fue modificado y que proviene del usuario esperado (Hexnode, s.f.).



9. Flujo criptográfico del envío de archivos



10. Componente matemático

10.1. X25519 y el secreto compartido

X25519 se basa en criptografía de curvas elípticas. La idea principal es que dos partes pueden generar un secreto compartido sin transmitirlo directamente. Cada usuario tiene una llave privada y una llave pública derivada matemáticamente.

El emisor combina su llave privada efímera con la llave pública del receptor. El receptor combina su llave privada con la llave pública efímera del emisor. Ambas operaciones producen el mismo secreto compartido.

La seguridad del mecanismo se basa en la dificultad de recuperar la llave privada a partir de la llave pública, problema relacionado con el logaritmo discreto en curvas elípticas.



10.2. AES-GCM e integridad autenticada

AES es un algoritmo de cifrado simétrico por bloques. En el modo GCM, el sistema no solo cifra la información, sino que también genera una etiqueta de autenticación. Esta etiqueta depende del texto cifrado, el nonce y los datos asociados.

Si un atacante modifica cualquier parte del paquete, la etiqueta calculada durante el descifrado ya no coincide. En consecuencia, el sistema rechaza el archivo y evita entregar información alterada como válida.

11. Modelo de amenazas

Elemento	Descripción
Activos	Archivos sensibles, contraseñas, llaves privadas, llaves públicas, identidades y logs.
Adversarios	Atacante externo, usuario malicioso, servidor comprometido o atacante con acceso a la red.
Capacidades	Capturar tráfico, modificar paquetes, reenviar mensajes, intentar contraseñas, robar la base de datos.
Objetivos del atacante	Leer archivos, modificar contenido, suplantar usuarios, repetir mensajes válidos o extraer credenciales.
Controles implementados	AES-GCM, Ed25519, X25519, Argon2, bloqueo de cuentas, nonces únicos y logs.
Riesgos residuales	Robo de llave privada, dispositivo infectado, ausencia de certificados formales o canal sin TLS.

12. Pruebas funcionales

12.1 Registro de usuario

- 1) El usuario primero se debe de registrar y posteriormente iniciar sesión.



Sistema de Documentos Seguros

Confidencialidad · Integridad · Autenticidad · Anti-repetición

Iniciar sesión **Registrar usuario**

Nuevo usuario

GabrielHC

Contraseña

Confirmar contraseña

Rol

Empleado

Crear cuenta

Sistema de Documentos Seguros

Confidencialidad · Integridad · Autenticidad · Anti-repetición

Iniciar sesión Registrar usuario

Usuario

GabrielHC

Contraseña

Ingresar al sistema

- 2) La pestaña que se ve enseguida es el panel. En este caso, el usuario ya había sido creado y tenía actividad.

Documentos Seguros

GabrielHC · Empleado

Panel de Control

Gestión de Archivos

Identidad y Llaves

Conexión Local Protegida

Cerrar sesión

Panel de Control

Sesión activa: GabrielHC (Empleado)

Archivos en tránsito (Hoy)

Eventos totales registrados

Firmas digitales

Actividad

3

56

2

8

Save your password?

Your password will autofill the next time and will be saved to your Microsoft account

Username

GabrielHC

Password

Save Not now

Deploy

- 3) Tabla mostrando hash y usuario

usuario	password_hash
admin	\$argon2id\$v=19\$m=65536,t=3,p=4\$Ccx4/XXYiQj1+7oZ/q+jJ5w\$FKSTWxiPLwXV8qIXwrMFj3wzUe1THXKw/yk9S35azBU
alice	\$argon2id\$v=19\$m=65536,t=3,p=4\$/iAMV0wprk7D+006nJtX6A\$HrKrRIYXEzsvrVuc2AaXvg0prwMd7gJWsvMohRRMs
bob	\$argon2id\$v=19\$m=65536,t=3,p=4\$y+MtV9c/Z75ohXh2r2WeRg\$8R61/xQ2FroG0ygyow2BG972FKAJpsL7qzYB+X7HQs8
so	\$argon2id\$v=19\$m=65536,t=3,p=4\$aptElwjw5V6Q8tVG/D65rQ\$cJovq1XsHeXsvXUcKr2FfTxar/CRXcxLogViuwkENeM
GabrielHC	\$argon2id\$v=19\$m=65536,t=3,p=4\$JK401H7woHDRAEynK/k0Jg\$cZchK53Vp/wvtv3KTCKMNSHR4TDGboUZrB/J+L7PXHA

12.2 Generación de llaves

- 4) El usuario recién creado no tiene llaves generadas



Documentos Seguros

GabrielHC - Empleado

☐ Panel de Control

☐ Gestión de Archivos

☒ Identidad y Llaves

Conexión Local Protegida

Cerrar sesión

Deploy

Identidad y Llaves

Gestión del material criptográfico asociado a su identidad.

[Ver llave pública](#) [Exportar llave privada](#) [Generar par de llaves](#) [Gestionar contraseña](#)

Su llave pública registrada

Aún no posee llaves vinculadas en el servidor público. Ve a la pestaña 'Generar par de llaves'.

- 5) El usuario puede generar su par de llaves que se protegerán si el usuario proporciona una “passphrase” o contraseña de la llave privada.

Identidad y Llaves

Gestión del material criptográfico asociado a su identidad.

[Ver llave pública](#) [Exportar llave privada](#) [Generar par de llaves](#) [Gestionar contraseña](#)

Generar un nuevo par de llaves reales

Este proceso generará tus identidades reales: Ed25519 para firmas y X25519 para cifrado.

Asigna una frase de seguridad para proteger tus llaves privadas

Generar e Inscribir Llaves

- 6) el usuario recibe su par de llaves privadas, una para firmar (Ed5519) y otra para cifrar (X25519)

Ver llave pública

Exportar llave privada

Generar par de llaves

Gestionar contraseña

Generar un nuevo par de llaves reales

Este proceso generará tus identidades reales: Ed25519 para firmas y X25519 para cifrado.

Asigna una frase de seguridad para proteger tus llaves privadas

Generar e Inscribir Llaves

Downloads

priv_cifrado_GabrielHC.pem

Open file

priv_firma_GabrielHC.pem

Open file

See more

Deploy

⚠ Descarga ambos archivos antes de cambiar de pestaña o cerrar sesión.

Descargar Llave Privada de Firma (Ed25519)

Descargar Llave Privada de Cifrado (X25519)



7) En la base de datos del sistema quedan guardadas las llaves públicas del usuario tanto para cifrar como para firmar.

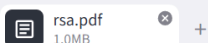
usuario	pub_x	pub_ed
bob	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAR+yjqZoc+gqTDJHQAxaigBYpZNXKH4SUhuJqTDainM= -----END PUBLIC KEY-----	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAti/iwHghV15fawKp7FERXZawt2ZUE173P3JKufmfZAK= -----END PUBLIC KEY-----
admin	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAPAHK1oeKK8vuzE8R53TUsKqjms0m7kfmeu5wQLiAXEQ= -----END PUBLIC KEY-----	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEA3swaj3enU2pmu6as4AX32ZlnypD3y8SsLcK+y2LY9oA= -----END PUBLIC KEY-----
alice	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAVqc/atB6LjYwX1ujHmgAtwq3xJ7X8mSwTFUFoU14XU= -----END PUBLIC KEY-----	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEA2KnI/1nxG0JHuVZRNjU13VpTChaX9shpZKHLdu0B1SM= -----END PUBLIC KEY-----
GabrielHC	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEABrboz30FdG6vrAU6xe4hyCbWErHJ8Jb6/AWAKjFQ= -----END PUBLIC KEY-----	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAtJYEX1zcehx11fbQo2wkoXt45S9P5AmwLVZ8iH7KQXc0= -----END PUBLIC KEY-----

12.3 Envío de archivo

El usuario para poder enviar un archivo necesita firmarlo por lo que necesita ingresar su llave privada con su contraseña o “passphrase” de la llave privada.

Enviar archivo cifrado

Seleccione el archivo a proteger



Destinatario

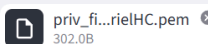
alice

Flujo de protección REAL

- Se firma el archivo original usando tu llave Ed25519.
- Se genera un secreto compartido mediante X25519.
- Se cifra con AES-256-GCM.
- Se empaquetan Nonces y marcas de tiempo.

Requisito de Firma: Sube tu Llave Privada de Firma

Tu llave privada de firma (priv_firma_...pem)



Contraseña de tu llave privada

Cifrar, Firmar y Enviar



12.4 Recepción y descifrado

El receptor del archivo tiene que ingresar su llave privada de cifrado y contraseña de la llave privada o “passphrase” para poder descifrar y verificar el archivo recibido.

rsa.pdf

De: GabrielHC | Enviado: 2026-07-14T21:59:36.160514

Tu llave privada de cifrado

priv_ci..._alice.pem
302.0B

+

Contraseña de la llave

.....

Descifrar y Verificar

¡Autenticidad e Integridad confirmadas!

Guardar Archivo Descifrado

12.5 Archivos enviados

- 1) Se muestran los archivos enviados en conjunción de la fecha, el nonce usado para el cifrado simétrico AES-GCM, el ID y el tamaño en bytes del mensaje.

Documentos Seguros

GabrielHC · Emp·Leado

Panel de Control

Gestión de Archivos

Identidad y Llaves

Conexión Local Protegida

Cerrar sesión

Gestión de Archivos

Envío cifrado, recepción con descifrado y verificación de firmas digitales.

Enviar archivo Recibir y descifrar **Archivos enviados** Verificar firma digital

Archivos enviados

Historial de archivos que has cifrado, firmado y enviado a otros usuarios.

ID	Destinatario	Archivo	ID único / Nonce	Fecha de envío	Nonce AES-GCM (hex)	Tamaño cifrado bytes
8	alice	rsa.pdf	44731846052af562c8b6b010844633c2	2026-07-14T23:48:05.625640	a287139c95ea5c77cbe5c64f	1100640

Detalle de envíos

> rsa.pdf → alice



2) se muestran los archivos enviados del usuario alice

Documentos Seguros

alice · Empleado

Panel de Control

●

Gestión de Archivos

Identidad y Llaves

Conexión Local Protegida

Cerrar sesión

Gestión de Archivos

Envío cifrado, recepción con descifrado y verificación de firmas digitales.

Enviar archivo Recibir y descifrar **Archivos enviados** Verificar firma digital

Archivos enviados

Historial de archivos que has cifrado, firmado y enviado a otros usuarios.

ID	Destinatario	Archivo	ID único / Nonce	Fecha de envío	Nonce AES-GCM (hex)	Tamaño cifrado byt
6	bob	car.jpg	cff817ebb2cd5d3c21a52ffdd388356c	2026-07-14T02:39:44.179441	f4a4bdc5491ebf18595ca6c8	497
5	bob	mensaje.txt	f623d867c55883830b81d414d9a34d92	2026-07-14T02:32:52.818102	c8364891ed16c2cf9df1416c	
4	bob	mensaje.txt	0e224feff710a995af17a935b6088251	2026-07-14T02:31:48.051265	4c22ac2df15ead0ba2f8d0b0	
3	bob	mensaje.txt	b8e61440c388751fd420a2bb485b4eb7	2026-07-14T02:31:46.368706	f9f0ed85e3d645d6ecbca4b0	
2	bob	mensaje.txt	cd05c241d9c301cdc2119b313b8ec1f4	2026-07-14T02:31:40.591249	31f7ba062638cacfb1de1dc3	
1	bob	car.jpg	02835dcfa767814c2ba4e8a5d3e4a8da	2026-07-12T02:59:18.739264	c860c475c3e3e84ae2f54215	497

13. Pruebas de Ataques

13.1. Modificación de Archivos (Integridad y Autenticidad)

Se simula que se intercepta un archivo donde un atacante (Man-in-the-Middle) altera uno o más bytes del documento con el fin de corromper la información o inyectar datos incorrectos con la intención de cometer fraude y comprometer la integridad.

Para contrarrestar esto se aplicó el Cifrado Simétrico Autenticado (AEAD). El sistema utiliza el algoritmo AES-256-GCM a través de la librería `crypto.py`. GCM no solo cifra los datos, también proporciona autenticidad al texto. Al momento de cifrar el algoritmo genera un código único de integridad conocido como tag, la cual se empaqueta junto con el archivo cifrado.

Para validar, durante el proceso de recepción, el método `aesgcm.decrypt()` realiza la verificación matemática de esta etiqueta antes de proceder a la reconstrucción del archivo original. Si un atacante modifica incluso un solo bit del archivo cifrado o de sus metadatos de control la validación del tag fallará automáticamente.

El programa lanzará internamente la excepción `InvalidTag`, la cual es interceptada por el backend para suspender la operación inmediatamente y lanzar un error controlado: `ValueError: "FALLO_DESCIFRADO: El archivo ha sido manipulado o la etiqueta de autenticación es inválida."`

13.2. Firma Falsa (Autenticidad de Origen y No Repudio)

Cuando el receptor del archivo descifra su contenido, se pueden descargar dos archivos, el original llamado `nombre_del_archivo.bin` y el de la firma llamado `nombre_del_archivo.bin.sig`. Con ambos se puede verificar en el sistema la legitimidad de la firma en la pestaña de Verificar Firma Digital.



Para esto se usó el esquema de Firmas Ed25519. El emisor original firma el hash del contenido claro utilizando criptografía asimétrica de curva elíptica para garantizar que solo el que tenga la llave privada sea también el que haya generado la firma digital.

El sistema no confía en firmas provistas de forma dinámica por el cliente. En su lugar, el backend consulta la tabla *llaves_publicas* de la base de datos para recuperar la clave pública oficial asociada al remitente. El receptor ejecuta la validación insertando el contenido descifrado del archivo con la firma adjunta y seleccionando el emisor del mensaje.

Si el archivo fue firmado con una clave privada distinta a la registrada, el emisor no coincide, o si el documento original fue alterado el sistema lanzará la siguiente advertencia: *ValueError: "FIRMA_INVALIDA: La firma digital no corresponde al emisor o al archivo enviado."* Adicionalmente, el sistema registrará un log de auditoría con prioridad de ERROR bajo el evento *"FIRMA_INVALIDA"*, alertando sobre un posible intento de suplantación de identidad en la plataforma.

13.3. Ataque de Repetición (Replay Attack)

En este ataque, el atacante intercepta un paquete de datos cifrado legítimo en tránsito y sin necesidad de descifrarlo ni alterar su firma, lo reenvía de manera repetida al servidor destino para forzar operaciones duplicadas.

Cada vez que se empaqueta un archivo en *crypto.py*, el sistema genera un identificador único aleatorio de 16 bytes que actúa como un *nonce* y se registra en el sistema como un token que sólo puede ser utilizado una vez.

Entre las restricciones a nivel de base de datos está que en el esquema de datos (*database.py*), la tabla destinada a almacenar el historial de envíos (*paquetes_archivos*) define la columna *nonce_protocolo* bajo la restricción UNIQUE NOT NULL.

Al procesar un paquete entrante, la función *guardar_paquete_archivo* intenta insertar el registro en la base de datos. Ahora, al ejecutar el simulador de ataque de repetición (reenvío de un paquete ya existe en el sistema se disparará de manera automática una excepción de integridad relacional (*sqlite3.IntegrityError*), provocando que la base de datos aborte la inserción y retorne un estado de fallo (*False*). El sistema detendrá el procesamiento del archivo duplicado y registrará una entrada crítica de auditoría en los logs del sistema: *registrar_log("CRITICAL", "INTENTO_REPETICION", ...)*

13.4. Contraseña Incorrecta (Mitigación de Fuerza Bruta)

Un actor malicioso ejecuta ataques automatizados de diccionario o de fuerza bruta para adivinar las contraseñas de las cuentas de usuario, realizando solicitudes de inicio de sesión continuas. El sistema protege las credenciales utilizando Argon2id,



el cual es el algoritmo ganador del Password Hashing Competition gracias a su alta resistencia a ataques.

La base de datos mantiene un control estricto del estado de seguridad de cada usuario mediante dos columnas en la tabla usuarios: *intentos_fallidos* y *estado*. Igualmente se definió un umbral de tolerancia que permite un máximo de 5 intentos, después de eso la cuenta se bloquea y se debe ir al usuario administrador para desbloquearla. Esto genera un log en la base de datos de la auditoría.

13.5. Robo de la Base de datos

Si es que por algún error de accesos o incluso si interfieren físicamente con el servidor para un hipotético robo de la base de datos, el o los atacantes no podrían tener acceso a la información sensible de manera sencilla, ya que:

- las contraseñas se guardan por medio de un hash robustecido a través del algoritmo argon2 que proporciona una robustez superior que simplemente usar el hash de la contraseña, siendo que aumente su resistencia a wordlists, fuerza bruta, etc, ya que usa llaves derivadas para este “hash”.

Como se mencionó anteriormente el atacante podría atacar a los hashes con wordlists, fuerza bruta, etc pero no sería efectivo ya que no se usa una simple función hash para obtener el valor guardado.

Información pública que podría conocer

- datos de usuarios

Tabla de Usuarios

Busca por nombre de usuario (0)

usuario	password_hash	rol	estado	intentos_fallidos	ultimo_acceso
admin	\$argon2id\$v=19\$m=65536,t=3,p=4\$4gl/pEBGHuDHwZWTKDFWSQ\$nuipGgEbuxWXFy	Superusuario	Activa	0	2026-07-15 23:17:34
bob	\$argon2id\$v=19\$m=65536,t=3,p=4\$Y8JOz1Np85edUv3QVzMcRg\$VQu/k7AdKL2Ulu8C	Empleado	Activa	0	2026-07-15 02:21:23
alice	\$argon2id\$v=19\$m=65536,t=3,p=4\$CJ1+6zEsmz7WGczBcM0upg\$I0a7mbfqIBrWJ/OF4	Empleado	Activa	0	None

- datos de llaves públicas



Tabla de Usuarios y Llaves Públicas

Busca por nombre de usuario (1)

pub_x	pub_ed
-----BEGIN PUBLIC KEY----- MCowBQYDK2VuAyEA/2pPmtlPmOWnGZje6NGK5RZdpZLS	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAj9RJY3NOIo1H+IFNfg/jrO
-----BEGIN PUBLIC KEY----- MCowBQYDK2VuAyEAzeESqol8bkmqQ0mulzoZB+EHvtJaM	-----BEGIN PUBLIC KEY----- MCowBQYDK2VwAyEAjHh1VwujjLHFqCcDIK970

- logs

Tabla de Logs

Busca por nombre de usuario (3)

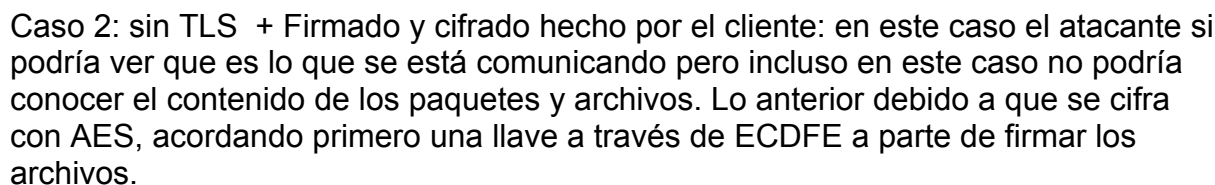
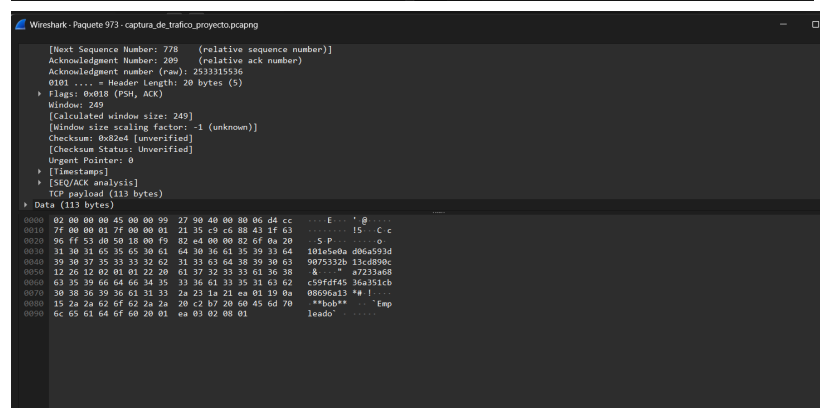
id	fecha_hora	nivel	evento	usuario	detalle
4b625470	2026-07-15 01:52:05	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
a447a99f	2026-07-15 01:52:05	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
e53037e5	2026-07-15 01:52:05	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
d8642645	2026-07-15 01:52:06	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
704e825d	2026-07-15 01:52:06	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
640f287c	2026-07-15 01:52:06	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
a13d54a6	2026-07-15 01:52:06	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
baab1637	2026-07-15 01:52:06	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
8d3581a0	2026-07-15 01:52:07	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
9c2da55e	2026-07-15 01:52:07	WARNING	LOGIN_FALLIDO	admin	Intento de acceso denegado: El usuario no existe.
-----	-----	-----	-----	-----	-----

- paquete

13.6. Ataque Man In the Middle

Este ataque se da cuando un atacante se logra infiltrar en las comunicaciones del servidor y cliente pudiendo observar la información que es comunicada, corromperla o modificarla. En este caso se expondrá qué información sensible puede quedar expuesta si es que ocurriera este suceso.

Caso ideal: uso de TLS + Firmado y cifrado hecho por el cliente: en este caso ideal un ataque Man in the Middle sería inútil ya que la comunicación está cifrada aparte de que no se exponen las llaves privadas ya que se usan desde el cliente.

[illegible]

Se puede ver que el usuario se ha autenticado como "bob" con el rol de "Empleado"

¿Qué información permanece cifrada?

Contenido de archivo, Secreto X25519 (nunca es transmitido)



¿Qué metadatos siguen siendo visibles?

- IPs, puertos, timestamps, nombre

¿Qué sucede cuando se reenvía un mensaje (REPLAY)?

Cuando un atacante intenta reenviar un paquete válido que ya fue procesado, el sistema tiene una protección específica: cada paquete tiene un identificador único llamado `nonce_protocolo` que nunca debe repetirse. La base de datos tiene una restricción `UNIQUE` en este campo, lo que significa que cuando el atacante reintentará enviar el mismo paquete con el mismo `nonce`, la base de datos automáticamente rechaza la inserción porque ese `nonce` ya existe.

¿Qué ocurre cuando se modifica un paquete o archivo?

El servidor envía un mensaje que dice: "El receptor intentó verificar la firma legítima del emisor usando el texto manipulado ('HOLA AAAAAA...')."

Lo que se ve en Wireshark es que el servidor está respondiendo con este mensaje que indica que alguien intentó enviar un archivo con el contenido modificado. El mensaje menciona específicamente que cuando el servidor intentó verificar que el archivo fuera auténtico usando la firma, el contenido que recibió no coincidía con lo esperado porque había sido alterado.

Qué cambia si se usa TLS

En lugar del contenido legible que se ve actualmente, solo aparecerían los datos encriptados, que se mostrarían como bytes aparentemente aleatorios sin patrón legible. Los mensajes "Ataque Replay", "texto manipulado", los headers HTTP, y todo el contenido que actualmente se puede leer pasarían a ser incomprensibles. La única información visible seguiría siendo las direcciones IP, los puertos y los números de secuencia TCP, pero el contenido de los datos transmitidos sería completamente ilegible.

Diferencia Transporte vs E2E

Mientras que el cifrado de la capa de transporte solo proporciona cifrado entre los proveedores de servicios y los usuarios individuales, el cifrado de extremo a extremo cifra las transmisiones de comunicación directamente entre los usuarios.

Con el cifrado de extremo a extremo, si envías un mensaje en texto plano, este se cifrará en tu extremo y solo se descifrá al llegar al ordenador del destinatario. Sin embargo, en el cifrado de capa de transporte, un mensaje en texto plano se cifra en el extremo del remitente y se descifra al llegar al servidor. En este punto, el mensaje solo se cifra si el destinatario también utiliza seguridad de capa de transporte, lo que lo convierte en una solución menos segura.



15. Parte post cuántica

La criptografía postcuántica busca proteger los sistemas actuales frente a computadoras cuánticas suficientemente potentes. Este tema es relevante porque algoritmos de llave pública como RSA, Diffie-Hellman y criptografía de curvas elípticas podrían ser vulnerables ante ataques cuánticos. NIST señala que los sistemas actuales basados en criptografía de llave pública deben migrar gradualmente hacia algoritmos postcuánticos (NIST, s.f.)

En el sistema desarrollado, los componentes más vulnerables ante un escenario postcuántico serían X25519 y Ed25519, ya que ambos pertenecen a la criptografía de curvas elípticas. X25519 se usa para establecer el secreto compartido entre emisor y receptor, mientras que Ed25519 se usa para firmar y verificar archivos. Por ello, en una versión futura del sistema estos mecanismos deberían reemplazarse por alternativas postcuánticas.

En cambio, AES-GCM y Argon2 seguirían siendo razonablemente seguros, ya que no dependen de factorización ni de logaritmos discretos. El cifrado simétrico con llaves suficientemente largas, como AES-256, sigue considerándose una opción viable frente al escenario postcuántico.

Para reemplazar X25519, se podría utilizar ML-KEM, estandarizado por NIST en FIPS 203. ML-KEM es un mecanismo de encapsulamiento de llaves diseñado para establecer secretos compartidos y se considera resistente frente a adversarios con computadoras cuánticas (NIST, 2024). Para reemplazar Ed25519, se podría utilizar ML-DSA, estandarizado por NIST en FIPS 204, el cual permite generar y verificar firmas digitales postcuánticas (NIST, 2024).

La diferencia principal es que ML-KEM sirve para establecer una llave compartida, mientras que ML-DSA sirve para firmar y verificar mensajes o archivos (NIST, 2024). En este proyecto, ML-KEM sustituiría el intercambio de llaves con X25519, y ML-DSA sustituiría la firma digital con Ed25519.

También debe considerarse el riesgo/dicho conocido como “cosecha ahora, descifra después”, donde un atacante captura información cifrada actualmente para intentar descifrar en el futuro cuando existan computadoras cuánticas más potentes. Por esta razón, organismos como NIST recomiendan iniciar la planeación de migración postcuántica desde ahora.

Finalmente, el sistema debería diseñarse teniendo en mente una respuesta ágil y adaptable, es decir, con la capacidad de cambiar algoritmos criptográficos sin rediseñar toda la aplicación. Para lograrlo, sería necesario registrar qué algoritmo se usa en cada paquete, manejar versiones de protocolo y separar el módulo criptográfico del resto del sistema.



17. Conclusiones

El sistema desarrollado demuestra la aplicación práctica de mecanismos criptográficos modernos para proteger documentos sensibles en una red no confiable. La combinación de AES-GCM, X25519, Ed25519 y Argon2 permite cubrir los objetivos de confidencialidad, integridad, autenticidad y protección de credenciales.

Además, el uso de identificadores únicos, timestamps y logs de auditoría permite detectar eventos relevantes de seguridad, incluyendo intentos de repetición, fallos de descifrado, firmas inválidas y accesos no autorizados.

El proyecto muestra que la seguridad no depende de un único mecanismo, sino de la integración correcta de autenticación, cifrado, firma digital, control de llaves, auditoría y análisis de amenazas. Aunque existen mejoras posibles para un entorno productivo, la implementación cumple con los objetivos académicos y demuestra un flujo funcional de intercambio seguro de archivos.



18. Referencias bibliográficas

JumpCloud. (2025). *What Is Argon2?* <https://jumpcloud.com/it-index/what-is-argon2>

Medium. (2024). *Understanding AES Encryption and AES/GCM Mode: An In-Depth Exploration using Java.* <https://medium.com/@pravallikayakkala123/understanding-aes-encryption-and-aes-gcm-mode-an-in-depth-exploration-using-java-e03be85a3faa>

Hansen, T. & Harrison, J. (2024). *Better-performing “25519” elliptic-curve cryptography.* <https://www.amazon.science/blog/better-performing-25519-elliptic-curve-cryptography>

Hexnode. (s.f.). *What is Ed25519 in cybersecurity?* <https://www.hexnode.com/blogs/explained/what-is-ed25519-in-cybersecurity/>

NIST. (s.f.) *Frequently Asked Questions about Post-Quantum Cryptography.* <https://pages.nist.gov/nccoe-migration-post-quantum-cryptography/FAQ/index.html>

NIST. (2024). *Module-Lattice-Based Key-Encapsulation Mechanism Standard: FIPS 203.* National Institute of Standards and Technology. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf>

NIST. (2024). *Module-Lattice-Based Digital Signature Standard: FIPS 204.* National Institute of Standards and Technology. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf>

Galaxkey. (s. f.). *What’s the difference between transport-layer encryption and end-to-end encryption?* <https://www.galaxkey.com/transport-layer-encryption-vs-end-to-end-encryption/>